Border Post Duty Roster

Viva Questions and Answers | Phase 1 and Phase 2 topics

Question bank based on the current CA2 C++ source file. Answers are kept short for oral revision.

Project overview

1. What is the purpose of this project?

Answer: It manages border post duty shifts. A user can add, view, search, read, update and summarize records.

2. What information does each shift contain?

Answer: A shift ID, post name, officer name, shift hours and strength.

3. What happens when the program starts?

Answer: loadAll() tries to read saved records from roster.bin into the array.

4. What happens when the program exits?

Answer: The program ends. Added and updated records have already been saved in the binary file.

5. What choices are available in the menu?

Answer: Add a shift, view shifts, search by ID, read from file, update, show summary or exit.

Class and data

6. What is a class?

Answer: A user-defined type that groups data and functions together.

7. What is the DutyShift class for?

Answer: It represents one duty shift and provides functions to update, display and convert its data.

8. What does private mean?

Answer: Private members can be accessed directly only by the class and its friends.

9. Why are the data members private?

Answer: To protect the shift data from direct changes outside the class.

10. What does public mean?

Answer: Public members can be used by code outside the class.

11. What is an object?

Answer: An instance of a class. Each element in shifts is a DutyShift object.

12. What is the Record structure for?

Answer: It stores fields in a fixed-size layout that can be written to or read from a binary file.

13. Why does Record use char arrays instead of string?

Answer: The program writes the whole Record as bytes. Fixed-size arrays give the record a predictable layout; a string object should not be written as raw binary this way.

14. What does FILE_NAME contain?

Answer: The string roster.bin, which is the binary data filename.

15. Why is FILE_NAME a const string?

Answer: It stores the filename as a C++ string, and const prevents changing it.

16. What does static int counter mean?

Answer: There is one shared counter for the class, not a separate counter for each shift.

17. How does the program assign a new ID?

Answer: The parameterized constructor increments counter and assigns it to shiftID.

18. Why is counter initialized outside the class?

Answer: A static data member needs one definition outside the class. Here it is initialized to zero.

19. What does DutyShift::counter = 0 do?

Answer: It defines and initializes the class static counter.

Constructors and destructor

20. What is a constructor?

Answer: A special class function that runs when an object is created.

21. What is the default constructor used for?

Answer: It initializes an empty shift with ID and strength zero and empty strings.

22. What does the parameterized constructor do?

Answer: It creates a shift using entered values and assigns a new ID.

23. Why does DutyShift shifts[MAX] call the default constructor?

Answer: The array creates 100 DutyShift objects, so the default constructor runs for every element.

24. What is an initializer list?

Answer: A way to initialize class members before the constructor body runs, such as shiftID(0).

25. What is a copy constructor?

Answer: A constructor that creates an object by copying another object of the same class.

26. Where is the copy constructor used?

Answer: updateRecord() creates oldShift from shifts[index] to display the existing record.

27. Why does the copy constructor copy the ID?

Answer: The copied object represents the same existing shift, so it keeps that shift ID.

28. What is a destructor?

Answer: A special function that runs when an object lifetime ends.

29. What does the destructor do here?

Answer: Its body is empty. It is included as a Phase 2 topic; the class does not manage memory manually.

Getters, friend function and pointers

30. What is a getter?

Answer: A function that returns the value of a private data member.

31. Why use getters?

Answer: They let other functions read private data without accessing it directly.

32. What does getID() return?

Answer: The shift ID.

33. What is inline about the getters?

Answer: Functions defined inside the class are implicitly inline.

34. What is a friend function?

Answer: A function outside the class that is allowed to access private members.

35. Why is summaryReport() a friend?

Answer: It reads the private strength value in each shift.

36. What does the summary report calculate?

Answer: It displays the number of shifts and the sum of their strengths.

37. What is a pointer to an object?

Answer: A variable that stores the address of an object. DutyShift *ptr is such a pointer.

38. How does searchByID() use a pointer?

Answer: When it finds an ID, it stores the matching object address in ptr and calls ptr->display().

39. What does the -> operator mean?

Answer: It accesses a member through a pointer; ptr->display() is equivalent to (*ptr).display().

40. What does NULL mean here?

Answer: It indicates ptr does not currently point to a matching object.

Strings and input

41. Why include string?

Answer: The code uses the C++ string class for names, hours and the filename.

42. Why use getline()?

Answer: It reads a full line, including spaces, such as a multi-word post or officer name.

43. Why call cin.ignore() before getline()?

Answer: It removes the leftover newline from the previous numeric input.

44. What does cin >> str do?

Answer: It reads strength as an integer.

45. What are the character-array limits in Record?

Answer: Post and officer can hold 29 visible characters each; hours can hold 19, leaving space for the ending null character.

Binary file handling

46. What is a binary file?

Answer: A file that stores data as bytes rather than as human-readable text.

47. What is fstream?

Answer: A C++ file-stream class used for file input and output.

48. Why include fstream?

Answer: The program uses fstream to read and write roster.bin.

49. What does ios::binary do?

Answer: It opens the file in binary mode.

50. What does ios::out mean?

Answer: It opens a file for writing.

51. What does ios::in mean?

Answer: It opens a file for reading.

52. What does ios::app mean?

Answer: It makes new output go to the end of the file.

53. How does appendToFile() save a shift?

Answer: It converts the shift to Record and writes its bytes at the end of roster.bin.

54. What does file.write() do?

Answer: It writes a specified number of bytes to the file.

55. What does sizeof(record) return?

Answer: The number of bytes occupied by the Record structure.

56. Why use reinterpret_cast<char *> with read and write?

Answer: Those functions take a character pointer, so the Record address is treated as bytes.

57. What does memset() do in toRecord()?

Answer: It sets all bytes of Record to zero before the fields are filled.

58. Why does toRecord() use c_str()?

Answer: It gives a string contents as a C-style character sequence for copying into a char array.

59. Why does toRecord() use strncpy()?

Answer: It copies a limited number of characters into each fixed-size array.

60. What does fromRecord() do?

Answer: It copies values from a Record structure back into a DutyShift object.

61. What does file.close() do?

Answer: It closes the file after the operation.

62. What does if (!file) check?

Answer: It checks whether opening the file failed.

63. What does seekp() do?

Answer: It moves the output position in the file so a record can be written at a selected location.

64. What does seekg() do?

Answer: It moves the input position so a record can be read from a selected location.

65. What does ios::beg mean?

Answer: The position is measured from the beginning of the file.

66. How is a record file position calculated?

Answer: The code multiplies the array index by sizeof(Record).

67. Why can the array index locate a file record?

Answer: Records are appended in array order and each Record has the same fixed size.

68. What does loadAll() do?

Answer: It reads saved records one by one into the array and updates count.

69. How does loadAll() restore the ID counter?

Answer: It finds the largest loaded ID and passes it to DutyShift::setCounter().

70. Why restore the counter after loading?

Answer: So the next shift gets an ID greater than the saved IDs.

71. What does DutyShift::setCounter(largestID) mean?

Answer: It calls the static function through the class and sets the shared counter.

Searching, updating and arrays

72. What is an array?

Answer: A group of values of the same type stored under one name.

73. What does DutyShift shifts[MAX] create?

Answer: An array that can hold 100 DutyShift objects.

74. What does count represent?

Answer: The number of array positions that contain valid shift records.

75. Why pass count by reference to addRecord() and loadAll()?

Answer: Those functions need to change the original count in main().

76. What does & mean in int &count?

Answer: It declares a reference, so the function uses the original integer instead of a copy.

77. What does findIndex() return?

Answer: The array position matching an ID, or -1 if it is not found.

78. Why return an index instead of the ID?

Answer: The index locates the object in the array and its corresponding file record.

79. How does searchByID() differ from readFromFile()?

Answer: searchByID() displays the object in memory. readFromFile() reads its corresponding record from the binary file.

80. How does updateRecord() work?

Answer: It finds the index, reads new details, updates the array object and overwrites the matching file record.

81. Does updating a shift change its ID?

Answer: No. update() changes only post, officer, hours and strength.

82. Why copy the old shift before updating?

Answer: It demonstrates the copy constructor and lets the program display the old details.

83. What if the requested ID is not found?

Answer: findIndex() returns -1 and the operation reports that the record was not found.

84. What if the roster array is full?

Answer: addRecord() displays a message and returns without adding a shift.

85. How does viewAll() display records?

Answer: It loops from zero to count - 1 and calls display() for each object.

86. Why does viewAll() check count == 0?

Answer: To show a message when there are no shifts to display.

Control flow and general concepts

87. What is a loop?

Answer: A control structure that repeats code.

88. What does the summary for loop do?

Answer: It adds each shift strength to totalStrength.

89. What does the do-while loop do in main()?

Answer: It shows and handles the menu at least once, then repeats until the user selects 7.

90. What does switch do?

Answer: It runs a different block based on the value of choice.

91. What does break do in a case?

Answer: It exits the switch after handling that choice.

92. What does return -1 mean in findIndex()?

Answer: It is a special value indicating that no matching record was found.

93. Why does main() return zero?

Answer: Zero conventionally indicates that the program finished successfully.

94. How does readFromFile() differ from loadAll()?

Answer: loadAll() reads every saved record at startup; readFromFile() reads one selected record.

95. Which modes are used when updating a file record?

Answer: ios::in, ios::out and ios::binary, so the file can be read and written in binary mode.

Limitations and follow-up questions

96. What if roster.bin does not exist at startup?

Answer: loadAll() cannot open it and returns, so the roster starts empty.

97. Can readFromFile() read an ID that is not in the array?

Answer: No. It first searches the in-memory array for the ID.

98. What if the file has more than 100 records?

Answer: loadAll() stops when the array reaches its capacity.

99. What if a name is longer than its Record array?

Answer: It is truncated when copied into the fixed-size array.

100. Does the program check every read and write result?

Answer: It checks whether the file opens, but does not separately report whether every read or write completed fully.

101. Why use a binary file here?

Answer: The project asks for binary file handling, and fixed-size Record objects can be written and read as blocks of bytes.

102. Which Phase 1 topics are in the program?

Answer: Class, static data member, inline getters, friend function, object array, object pointer and string class.

103. Which Phase 2 topics are in the program?

Answer: Default, parameterized and copy constructors; destructor; fstream binary file handling; seekg, seekp and startup reload.

104. Does the program implement Phase 3 or Phase 4?

Answer: No. It does not use inheritance, polymorphism, templates, exceptions or STL containers.

105. Why include cstring?

Answer: It provides memset() and strncpy(), used for the fixed-size fields in Record.

106. Give a short explanation of your project.

Answer: My project manages border post duty shifts. It stores each shift in an array and saves records in roster.bin. At startup it reloads records, and it can search, display and update them. It uses constructors and seekg/seekp for file access.